

# Day 3 lab: from a trained model to the microcontroller

Naeemullah Khan

[naeemullah.khan@kaust.edu.sa](mailto:naeemullah.khan@kaust.edu.sa)



جامعة الملك عبد الله  
للعلوم والتقنية  
King Abdullah University of  
Science and Technology

KAUST Academy  
King Abdullah University of Science and Technology

October 7, 2026

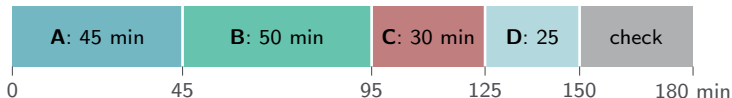
- ▶ **Goal:** your group has a motion classifier that runs on the XIAOML Kit, and you know its flash use, its arena size, and its latency.
- ▶ **Board:** the XIAOML Kit, with the Arduino firmware.
- ▶ **Deliverable:** a live demonstration of the four motions, and the file `report.md` with the comparison table of the two paths and the Decision Log.

## From the lecture to the lab

The lecture gave the format, the runtime, and the features. In the lab, you train a model on your Day 2 dataset, you convert it, you write the inference code, and you measure the three numbers on the board.

**Key point:** A number from the laptop is an estimate. Only the number from the board is a result.

# Plan of the three hours



| Part                    | Work                                                                        | Time   |
|-------------------------|-----------------------------------------------------------------------------|--------|
| A                       | Train: features of the Day 2 dataset, a small classifier                    | 45 min |
| B                       | Convert and deploy: LiteRT file, C array, sketch with TensorFlow Lite Micro | 50 min |
| C                       | Measure: flash, arena, latency, with PSRAM and with no PSRAM                | 30 min |
| D                       | Compare: the same task in Edge Impulse Studio                               | 25 min |
| Check by the instructor |                                                                             | 30 min |

Write each result in `report.md` when you get it.

## Hardware for each group

- ▶ XIAOML Kit with the Arduino firmware. If MicroPython is on the board, do the last step of the Day 2 lab first.
- ▶ USB-C data cable.
- ▶ Laptop with your Day 2 dataset in `Labs/day02/data/`.

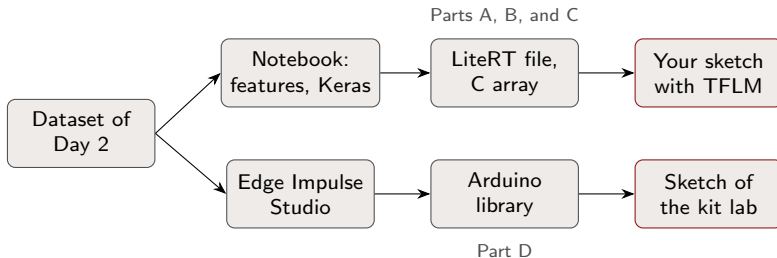
## Software

- ▶ Arduino IDE 2 with the esp32 core 3.3.12.
- ▶ New today: the library `Chirale_TensorFlowLite 2.0.0`.
- ▶ New today: the Python packages `tensorflow-cpu` and `ai-edge-litert`.
- ▶ Your Edge Impulse project of Day 2.

Lab files    `Labs/day03/`. Run the commands from this folder.

Install      In the environment of Day 1: `pip install tensorflow-cpu ai-edge-litert`

# Two paths to the same classifier

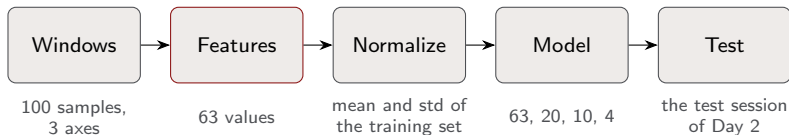


- ▶ The two paths use the same recordings, the same window of 2 s, and the same type of features: 63 values.
- ▶ In the first path, you write the inference code, and you see each step.
- ▶ In the second path, a cloud tool makes the features, the model, and the library.
- ▶ At the end, one table compares the two results on the board.

# Part A: train (45 min)

Start the notebook: `../../.venv/bin/jupyter lab motion_classifier.ipynb`. It uses your recordings in `Labs/day02/data/`.

| Task | You write                                            | The notebook checks                                                       |
|------|------------------------------------------------------|---------------------------------------------------------------------------|
| 1    | The function that removes the mean and gives the RMS | A sine wave with the amplitude 1 gives an RMS of 0.707                    |
| 2    | The two hidden layers of the model                   | The model has 1534 parameters                                             |
| 3    | Your prediction: features against the raw window     | Not checked. You compare it with the result.                              |
| 4    | The two lines of the converter (Part B)              | The file starts correctly, and its output is equal to the output of Keras |
| 5    | Your estimate of the arena size (Part B)             | Not checked. You measure it in Part C.                                    |



- ▶ **You see:** the plot of the spectral power for one window of each class. Find the axis and the frequency that separate your classes.
- ▶ **You record:** the training accuracy, the test accuracy, and the confusion matrix.

**Prediction (task 3).** The notebook trains a second model with the raw window: 300 inputs. How many parameters does it have? Which input gives the higher test accuracy? Write the prediction before you run section 6.

**Task 4.** Write the two lines of the converter in section 7 of the notebook. The notebook then prints lines of this form:

```
Task 4: complete
LiteRT file:          ... bytes
operator types:       [...]
largest difference between Keras and LiteRT: ...
```

- ▶ The difference must be smaller than 0.00001.
- ▶ **You record:** the file size and the operator types. Task B1 needs the operator types.

Section 8 writes three files into `sketches/motion_classifier/`:

---

|                               |                                                                          |
|-------------------------------|--------------------------------------------------------------------------|
| <code>model.h</code>          | The LiteRT file as a C array                                             |
| <code>model_settings.h</code> | The class names, and the mean and the standard deviation of each feature |
| <code>test_window.h</code>    | One test window, its 63 features, and the output of the model for it     |

---

**Task 5.** Estimate the arena size from the table of the last cell. Write the estimate in the report before Part C.

Install the library Chirale\_TensorFlowLite 2.0.0. Open `sketches/motion_classifier/motion_classifier.ino`. Set Tools > PSRAM > Disabled.

| Task | Place                     | You write                                                                                           |
|------|---------------------------|-----------------------------------------------------------------------------------------------------|
| B1   | <code>setupModel()</code> | The operator resolver: one <code>Add...()</code> call for each operator type of your model          |
| B2   | <code>setupModel()</code> | The interpreter, and the call of <code>AllocateTensors()</code>                                     |
| B3   | <code>classify()</code>   | Normalize the features, write the input tensor, call <code>Invoke()</code> , read the output tensor |
| B4   | <code>bestClass()</code>  | The position of the largest probability                                                             |

- ▶ The sketch compiles before you start. It then prints Tasks B1 and B2 are not complete.
- ▶ **You record:** the line `Sketch` uses ... bytes of this first build. It is the flash use with no runtime in the program.

Upload the complete sketch. The Serial Monitor at 115200 baud shows:

```
Arena location:      internal RAM
Model size (flash):  ... bytes
Sketch size (flash): ... bytes
Arena size:          4096 bytes
Arena used:          ... bytes
Self-test, features: largest difference ...
Self-test, model:    largest difference ...
Self-test, class:    ... (expected: ...)
Self-test:           PASS
```

- ▶ The self-test calculates the features of the window of `test_window.h` on the board, runs the model, and compares the results with the results of the notebook.
- ▶ FAIL: check task B3, and run section 8 of the notebook again.
- ▶ Then make each of the four motions. The display shows the class and its probability.
- ▶ **You record:** for each motion, the class that the board shows most of the time.

Source: The output shows the form of the lines.

# Part C: measure three numbers (30 min)

| Number  | Where you read it                                                        | You record                                                      |
|---------|--------------------------------------------------------------------------|-----------------------------------------------------------------|
| Flash   | Sketch uses ... bytes of the build, and Model size of the Serial Monitor | The two values, and the difference to the first build of Part B |
| Arena   | Arena used of the Serial Monitor                                         | The value, and your estimate of task 5                          |
| Latency | The columns features_us and invoke_us                                    | The median and the maximum of 20 lines                          |

One line of the Serial Monitor for each inference:

```
class,probability,features_us,invoke_us,late_samples  
.....
```

- ▶ The sum of the two times must be below 20 000 microseconds: one sample period.
- ▶ `late_samples` counts the samples that the loop read too late. Write the value after 20 lines.

## The arena size

1. Set `kTensorArenaSize` to 1024. Upload. Write the message of the Serial Monitor in the report.
2. Set it to the value `Arena` used plus 256 bytes. Upload. The self-test must pass.
3. Compare the measured value with your estimate. Explain the difference in one sentence.

## The second configuration

1. Set `Tools > PSRAM > OPI PSRAM`.
2. Change the line of the sketch to `#define ARENA_IN_PSRAM 1`.
3. Upload. The Serial Monitor prints `Arena` location: `PSRAM`.
4. Write the same three numbers in the second column of the report.

**Prediction.** Is the inference faster, slower, or equal when the arena is in the PSRAM? Write your answer and the reason before you measure.

1. **Impulse.** Open your project of Day 2. In Create impulse: window size 2000 ms, window increase 200 ms, the block Spectral Analysis, and the block Classification.
  2. **Features.** In Spectral features: FFT length 32. Generate the features.
  3. **Training.** In Classifier: two dense layers with 20 and 10 neurons, 30 training cycles. Then run Model testing.
  4. **Library.** In Deployment: Arduino library. Add the ZIP file with Sketch > Include Library > Add .ZIP Library.
  5. **Sketch.** Open sketches/ei\_motion\_inference/. Change the first `#include` line to the header of your library. Upload.
- ▶ **You record:** the test accuracy, and the estimates of the Studio for the inferencing time, the peak RAM, and the flash use.
  - ▶ **You record:** the two times of the line Predictions (DSP: ... ms, Classification: ... ms, ...) and the line Sketch uses ... bytes.
  - ▶ If the build of the library is not complete in time, use the estimates of the Studio.

| Number                 | Your sketch with TensorFlow Lite Micro | Library of Edge Impulse                |
|------------------------|----------------------------------------|----------------------------------------|
| Test accuracy          | Section 5 of the notebook              | Model testing                          |
| Flash                  | Sketch uses ... bytes                  | Sketch uses ... bytes, or the estimate |
| RAM                    | Arena used                             | The peak RAM of the Studio             |
| Time for the features  | features_us                            | DSP time                               |
| Time for the inference | invoke_us                              | Classification time                    |
| Kernels                | Reference kernels                      | Optimized kernels (ESP-NN)             |

- ▶ Write “measured” or “estimate” next to each number of the second column.
- ▶ The two paths use different units: microseconds and milliseconds. Write the unit.

The instructor checks this at the end of the lab:

- ▶ ✓ The notebook prints `complete` for tasks 1, 2, and 4.
- ▶ ✓ The Serial Monitor shows `Self-test: PASS`.
- ▶ ✓ The board shows the correct class for each of the four motions.
- ▶ ✓ The report has the three predictions: task 3, the arena estimate, and the PSRAM question. You wrote them before the measurement.
- ▶ ✓ The comparison table has measured numbers for your sketch: flash, arena, and latency, with PSRAM and with no PSRAM.
- ▶ ✓ The comparison table has numbers for the path of Edge Impulse. Each number says if it is a measurement or an estimate.
- ▶ ✓ The Decision Log gives numbers and names one trade-off.

**Hand in:** the file `report.md`. Show the kit to the instructor.

Write about 100 words. State one design decision, give your measured numbers, and name the trade-off.

A product must classify the motion of a package for one year with a battery. Which path do you select for the firmware?

- ▶ Your own sketch with TensorFlow Lite Micro.
- ▶ The library of Edge Impulse.

A good Decision Log:

- ▶ gives the flash, the RAM, and the latency of the two paths from your table,
- ▶ says what the latency means for the time that the processor can sleep,
- ▶ names what you lose with your decision. Example: control of the code against the speed of optimized kernels.

# If a part does not work, and common problems

| Problem                                         | Fix or fallback                                                                           |
|-------------------------------------------------|-------------------------------------------------------------------------------------------|
| Your group has no dataset of Day 2              | Copy the folder data/ of a different group into Labs/day02/. Write this in the report.    |
| The inference code is not complete in time      | Use the sketch of solutions/ with your three files of the notebook. Continue with Part C. |
| The upload fails after Day 2                    | MicroPython is on the board. Start bootloader mode, then upload.                          |
| Didn't find op for builtin opcode               | The resolver does not name an operator of the model (task B1).                            |
| AllocateTensors() failed                        | The arena is too small. Increase kTensorArenaSize.                                        |
| The self-test prints FAIL                       | Check the normalization in task B3. Run section 8 of the notebook again.                  |
| The board shows uncertain                       | The largest probability is below 0.6. Make the motion as in your Day 2 recordings.        |
| The board shows the wrong class for all motions | The model learned from the fallback dataset. Train with your own recordings.              |
| The library of Edge Impulse does not build      | Use the estimates of the Studio, and write "estimate".                                    |

The README of the lab has the complete list.

- ▶ *Machine Learning Systems* by Vijay Janapa Reddi and contributors, Harvard University. [mlsysbook.ai](https://mlsysbook.ai), CC BY-NC-SA 4.0.
- ▶ *TinyML Made Easy: XIAO ESP32S3* by Marcelo Rovai. [github.com/Mjrovai/XIAO-ESP32S3-Sense](https://github.com/Mjrovai/XIAO-ESP32S3-Sense), Apache-2.0.